

Graph-Based Question Answering System

Comparison of Graph RAG and Vector RAG for Multi-Hop Movie Queries

Purnima Nallamilli

2026

Introduction

Traditional RAG retrieves information using semantic similarity between a query and stored text, which works well for direct facts but struggles when a question requires connecting multiple related entities.

This project implements Problem Statement 3 – a Graph-Based Question Answering System – in the movie domain, comparing Graph RAG (Neo4j Knowledge Graph) and Vector RAG (ChromaDB) on the same dataset. The Knowledge Graph enables relationship-aware, multi-hop retrieval, while Vector RAG retrieves semantically similar chunks. Both pipelines use Google Gemini 2.5 Flash, with a Streamlit app running each question through both side by side.

Objectives

- Build a structured movie Knowledge Graph in Neo4j.
- Implement Graph RAG via natural-language-to-Cypher generation.
- Implement Vector RAG using ChromaDB and semantic embeddings.
- Evaluate both on the same benchmark questions and compare accuracy, speed, and completeness.

System Architecture

Two independent pipelines sit behind a common Streamlit interface. **Graph RAG** converts the question to a Cypher query, executes it against Neo4j, and generates the answer from the retrieved graph context. **Vector RAG** embeds the question, retrieves similar chunks from ChromaDB, and generates the answer from that context. Outputs from both are shown side by side with execution latency recorded.

Technology Stack

Python, Google Gemini 2.5 Flash, Neo4j, ChromaDB, LangChain, Google embedding models, Streamlit.

Graph Design

The Knowledge Graph contains 10 node types – **Movie**, **Actor**, **Director**, **MusicDirector**, **Writer**, **Editor**, **Cinematographer**, **Producer**, **ProductionHouse**, and **Genre** – connected through relationships such as **ACTED_IN**, **DIRECTED**, **COMPOSED_MUSIC_FOR**, **WROTE**, and **BELONGS_TO**. This lets multi-hop questions traverse several connected nodes rather than relying on a single

text chunk.

Retrieval Approach

Graph RAG: the question is converted into a Cypher query, executed against Neo4j, and the retrieved graph context is passed to Gemini for the final answer – well-suited to multi-hop questions.

Vector RAG: the question is embedded, ChromaDB retrieves the most similar chunks, and Gemini generates the answer from that context – effective for direct facts, but does not explicitly preserve relationships.

Comparison

Aspect	Graph RAG	Vector RAG
Storage	Neo4j	ChromaDB
Retrieval	Graph traversal	Semantic similarity
Multi-hop queries	Stronger	More challenging
Main strength	Relational reasoning	Fast semantic retrieval

Table 1: Comparison of the two retrieval approaches

Evaluation Results

Both pipelines ran the same benchmark of single-, two-, and three-hop questions. On the demonstrated relationship-intensive query:

Metric	Graph RAG	Vector RAG
Execution Latency	4.735 sec	3.458 sec
Retrieval Accuracy	90%	30%
Answer Accuracy	95%	20%

Table 2: Evaluation on the demonstrated relationship-intensive query

Vector RAG was faster, since Graph RAG performs query generation and traversal first. Graph RAG achieved substantially higher accuracy on this query, as its explicit structure let it follow the required relationships. These figures reflect this project’s own benchmark, not a universal comparison.

Observations and Trade-offs

Graph RAG performs strongly on multi-entity relationship questions since relationships are explicit in Neo4j, giving better structural explainability – though it needs careful schema design and Cypher generation adds overhead. Vector RAG has lower latency and suits straightforward factual questions, but struggles when the answer needs a relationship chain that similar chunks don’t preserve.

Graph RAG suits relationship-heavy, multi-hop questions; Vector RAG suits fast semantic retrieval of straightforward facts. Neither is universally better – it depends on the query’s structure.

Challenges Faced

- Designing a graph schema that clearly represents movie entities and relationships.
- Converting natural-language questions into valid Cypher queries.
- Keeping both pipelines on the same dataset for a fair comparison.
- Integrating Neo4j, ChromaDB, Gemini, LangChain, and Streamlit into one application.

Future Improvements

- Implement a true hybrid Graph + Vector retrieval strategy.
- Expand the dataset and add more multi-hop benchmark questions.
- Introduce Precision@K, Recall@K, F1-score, and support cloud deployment.

Conclusion

This project delivers a working Graph-Based QA system comparing Graph RAG with Vector RAG over the same movie-domain dataset, combining Neo4j, ChromaDB, Google Gemini 2.5 Flash, LangChain, and Streamlit. The results show Vector RAG offers lower latency, while Graph RAG delivers stronger accuracy on relationship-intensive questions – explicit relationships make it easier to reason across multiple connected entities. Graph RAG is most valuable when questions depend on multi-hop reasoning, while Vector RAG remains useful for fast retrieval of straightforward facts.